

АКОС 12

Статическая и динамическая линковка

Линковка и сборка проектов

- Статическая линковка
 - Таблица символов
 - Слабые символы
 - Создание статических библиотек
- Динамическая линковка
 - Создание динамических библиотек
 - Library Interpositioning

Что делает линкер?

- Ставит в соответствие каждому упоминанию символа ровно одно определение символа
- Сликает разделы кода и данных из всех translation unit в общие разделы
- Перемещает символы с относительных позиций в объектном файле в абсолютные позиции в исполняемом файле
- Редактирование всех ссылок на символы

Dump

Сначала в коде стоят заглушки ([rip + 0x0])

```
7 000000000000000000 <main>:
8 0: f3 0f 1e fa          endbr64
9 4: 55                    push    rbp
10 5: 48 89 e5             mov     rbp, rsp
11 8: 8b 05 00 00 00 00     mov     eax, DWORD PTR [rip+0x0]          # e <main+0xe>
12 e: 48 8d 15 00 00 00 00   lea     rdx, [rip+0x0]                    # 15 <main+0x15>
13 15: 89 c6                 mov     esi, eax
14 17: 48 8d 05 00 00 00 00   lea     rax, [rip+0x0]                    # 1e <main+0x1e>
15 1e: 48 89 c7             mov     rdi, rax
16 21: b8 00 00 00 00       mov     eax, 0x0
17 26: e8 00 00 00 00       call    2b <main+0x2b>
```

\$ objdump -d -M intel main.o

Dump

- После линковки стоят реальные адреса / смещения до объектов

```
160 00000000000011e9 <main>:
161    11e9:  f3 0f 1e fa      endbr64
162    11ed:  55              push    rbp
163    11ee:  48 89 e5         mov     rbp, rsp
164    11f1:  8b 05 19 2e 00 00 mov     eax, DWORD PTR [rip+0x2e19]      # 4010 <global_int>
165    11f7:  48 8d 15 12 2e 00 00 lea     rdx, [rip+0x2e12]      # 4010 <global_int>
166    11fe:  89 c6           mov     esi, eax
167    1200:  48 8d 05 fd 0d 00 00 lea     rax, [rip+0xdfd]      # 2004 <_IO_stdin_used+0x4>
168    1207:  48 89 c7         mov     rdi, rax
169    120a:  b8 00 00 00 00   mov     eax, 0x0
170    120f:  e8 8c fe ff ff   call    10a0 <printf@plt>
```

Статическая линковка

- **extern** int global_int; - **объявление** внешней (вне данной единицы трансляции) переменной
- **extern** void lib_foo(); - **объявление** внешней функции (по умолчанию и так extern)
- **static** int static_int; - **определение** статической переменной (видна только в текущей единице трансляции)
- Неинициализированные и инициализированные нулём глобальные переменные лежат в секции **.bss**
- Остальные – в секции **.data**
- Посмотреть таблицу символов: **\$ objdump -t a.out**

Статические библиотеки

- В момент компиляции (а именно линковки) встраиваются в исполняемый файл
- `$ gcc -c lib.c -o static.o` – собираем объектный файл
- `ar rc libSTATIC_LIB.a static.o` – упаковываем объектные файлы (обычно их много) в одну статическую библиотеку
- `gcc main.c libSTATIC_LIB.a` – компилируем и линкуем
- `gcc main.c -L. -lSTATIC_LIB` – указываем путь до библиотеки (-L – путь, где искать; -l – какую библиотеку линковать)

Другие способы указать путь

- `LD_LIBRARY_PATH` – переменная окружения, содержащая пути до библиотек
- `/etc/ld.so.conf` – конфиг с путями
- `ldconfig` – обновить пути из конфига выше

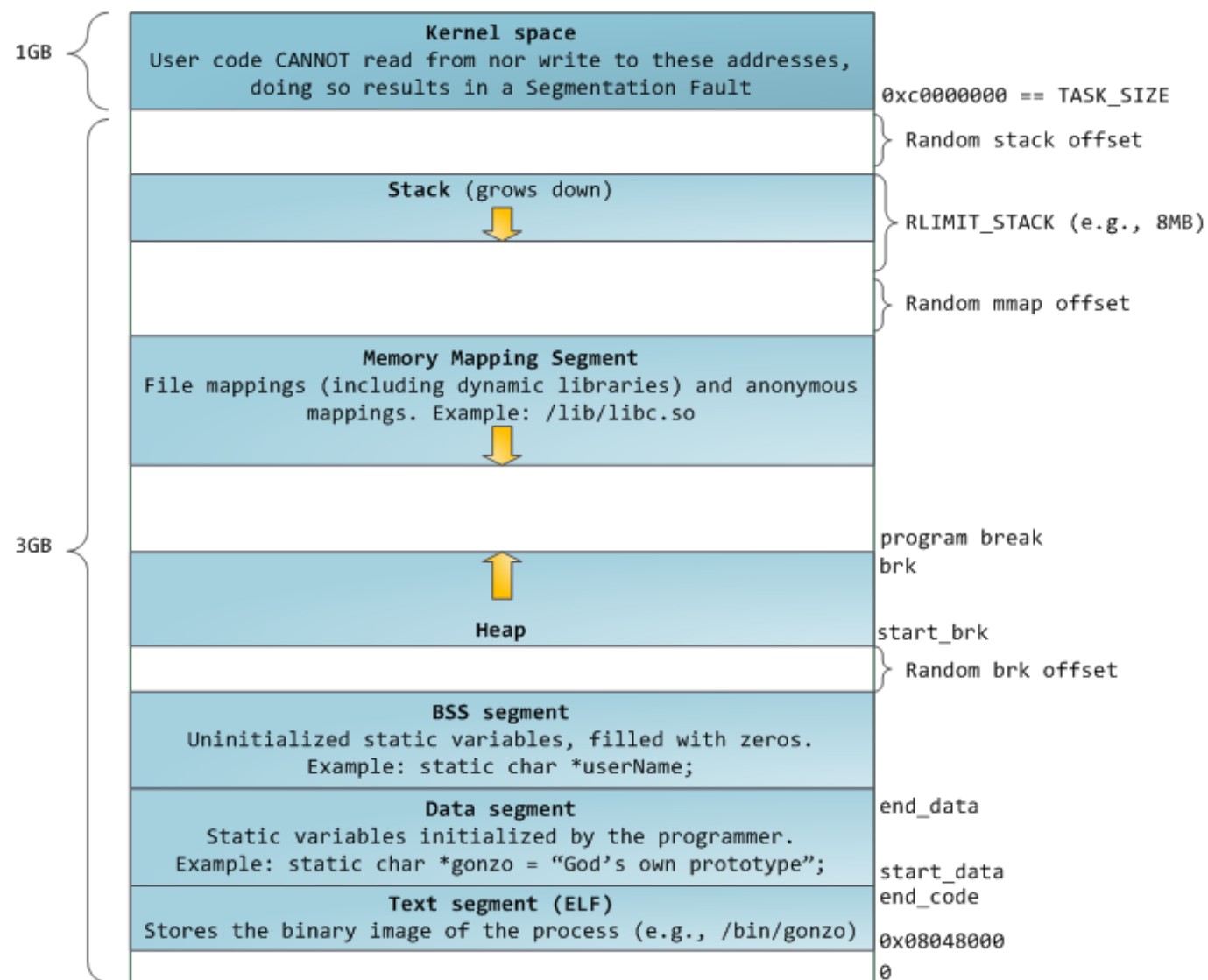
Сильные и слабые символы

- Слабые символы –
 1. `inline`
 2. `template`
 3. `__attribute__((weak))`
 4. `#pragma weak`
 5. неинициализированные глобальные переменные (иногда)
- Сильные символы – все остальные
- Правила разрешения символов:
 - У сильного символа может быть только одно определение (One Definition Rule)
 - Из нескольких определений символа выбирается сильное
 - Из нескольких слабых определений выбирается любое

Динамические библиотеки

- Подключаются к программе во время загрузки, начала исполнения, во время исполнения
- Позволяют не пересобирать всю программу
- Экономия места
- Имя: *.so
- ldd – позволяет узнать динамические библиотеки бинарного файла

Напоминание про адресное пространство



Создание динамической библиотеки

- `void _init()` – будет вызвана при загрузке в память
- `void _fini()` – будет вызвана при выгрузке из памяти
- `gcc -fPIC -c dynamic.c -o dynamic.o` компилируем библиотеку (с относительной адресацией, об этом далее)
- `gcc -shared -o libdynamic.so dynamic.o` – создать дин библиотеку

Подключение динамической библиотеки

- `gcc main.c -L. -ldynamic` – компиляция с указанием динамической библиотеки для сборки
- Указать пути в `$LD_LIBRARY_PATH` – для исполнения

Загрузка из кода программы

- `#include <dlfcn.h>`
- `void* dlopen(const char* filename, int flags);` - загружает дин библиотеку
- `void* dlsym(void* handle, const char* symbol);` - возвращает указатель на символ из библиотеки
- `int dlclose(void* handle);` - выгружает библиотеку из памяти
- Flags:
 - `RTLD_LAZY` – подгружать библиотеку лениво (при использовании)
 - `RTLD_NOW` – подгружать библиотеку моментально
 - ...
- `gcc -ldl main.c`

Загрузка из Python

```
import ctypes
```

```
dl = ctypes.cdll.LoadLibrary("libdynamic.so")
```

```
dl.hello()
```

Library interpositioning

- Перехват вызовов произвольных функций
- Пригодится для безопасности, наблюдения или профилирования
- Можно осуществить в момент компиляции, линковки, загрузки
- LD_PRELOAD – можно указать путь до динамической библиотеки, которая будет загружена до любых других библиотек (включая libc.so)
- /etc/ld.so.preload – можно использовать конфиг вместо переменной окружения

Library interpositioning

- Может быть, мы хотим вызвать оригинальную функцию
- В `dlsym` можно передать псевдо-обработчики:
 - `RTLD_DEFAULT` – находит первое вхождение символа
 - `RTLD_NEXT` – находит следующее вхождение (в нашем случае – изначальную функцию)

Полезные тулзы

- `$ readelf -h a.out` – посмотреть хэдер (заголовок) исполняемого файла
- `$ readelf -s a.out` – посмотреть таблицу символов
- `$ objdump -t a.out` – тоже посмотреть таблицу символов
- `$ objdump -d a.out` – dump исполняемого файла
- `$ nm -C a.out` – перечислить символы с demangling

Что почитать

- Про [Library interpositioning](#) (простенький руткит)
- Про [PIC, PLT и GOT](#)